Member-only story

Claude Code + | Anthropic Claude + | Productivity + | LLM +

Ai Knowledge Management +

LLM Wiki Skill: Build a Second Brain With Claude Code and Obsidian

Karpathy's wiki pattern compiles knowledge instead of retrieving it — here is the complete Claude Code skill I built to make it work in production



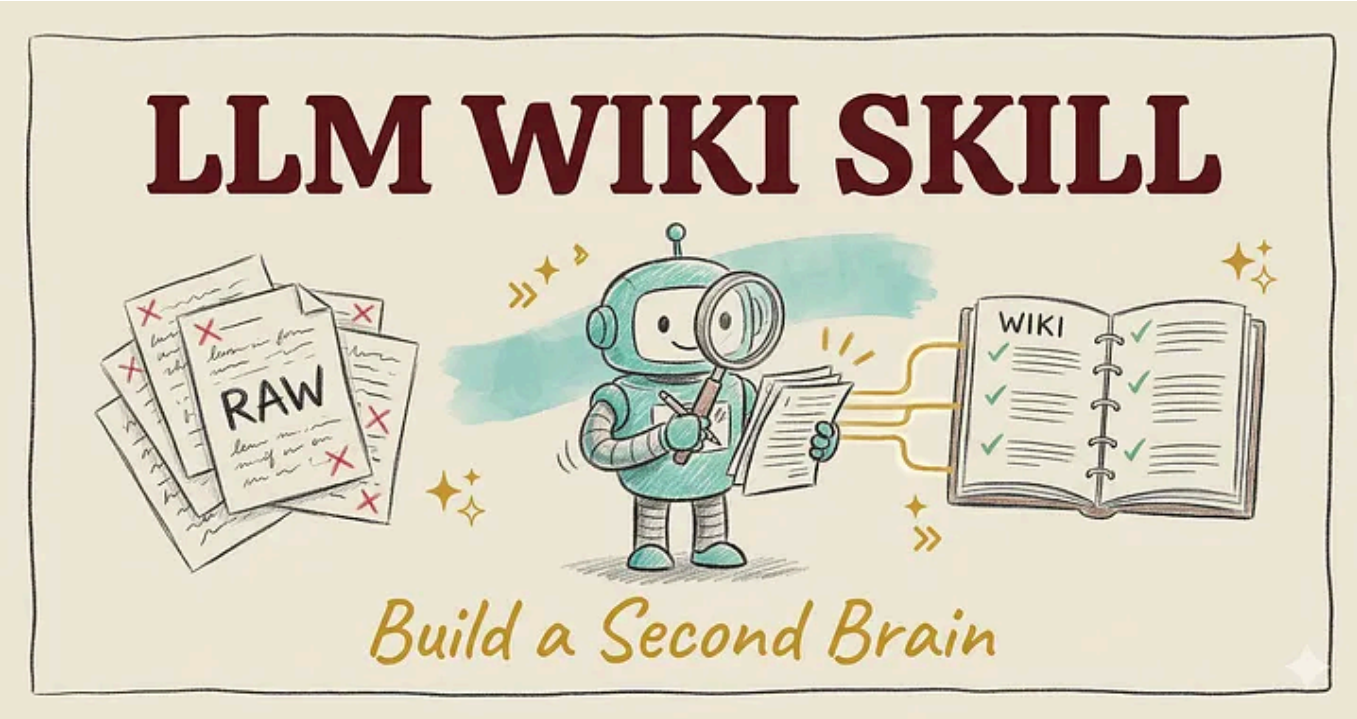
Reza Rezvani Follow 16 min read · Apr 12, 2026

274 5

Three months of architecture decisions lived in 14 teams threads, two Notion databases, a shared Google Doc that nobody updated after January, and my

own memory. Last Tuesday a new engineer asked why we chose event-driven messaging over REST for our notification service. I knew the answer.

I had debated it in a meeting, sketched it on a whiteboard, and defended it in a pull request comment. But finding the actual reasoning took twenty-five minutes of scrolling.



LLM Wiki Skill: Build a Second Brain | Image: Gemini Pro © Alireza Rezvani

Note: *AI tools were used during the editing process. The testing, technical analysis, and recommendations are based on my own experience.*

That same week, Andrej Karpathy published a gist titled *"LLM Wiki"* — 5,000 stars in four days, nearly 3,000 forks. One sentence in the gist reframed the entire problem: the shift is from retrieval to compilation. Not *"find the document that answers this query."*

Instead: *"build and maintain a persistent, cross-referenced knowledge base that already contains the synthesized answer."*

I have been writing about Karpathy's work since March. When he released autoresearch, I turned the optimization loop into a reusable Claude Code skill that works beyond ML training.

When he launched AgentHub, I wrote the practical setup guide with a working agent template. The LLM Wiki completes a trilogy: autoresearch teaches agents to optimize, AgentHub teaches agents to collaborate, and the LLM Wiki teaches agents to remember.

This article does three things.

First, it explains the core shift behind the pattern — why compilation beats retrieval.

Second, it walks through two use cases where the wiki actually changes how you work.

Third — and this is the part no other guide covers — it provides a complete, production-ready Claude Code skill you can install and start using today.

The Core Shift: Compilation Over Retrieval

The standard pattern everyone knows is RAG. Upload documents. The LLM finds relevant chunks at query time. It generates an answer. You ask another question. The LLM rediscovers everything from scratch.

Karpathy's metaphor is precise: this is like having a research assistant who reads every book in your library but forgets everything the moment you stop talking. You ask a question. He runs to the shelves. He comes back with an answer. You ask a follow-up. He runs again. He never connects anything.

The wiki pattern inverts this. When you add a new source, the LLM does not just index it for later retrieval. It reads the source, extracts the important

information, updates existing wiki pages, creates new ones, flags

Medium Search Play Games Write

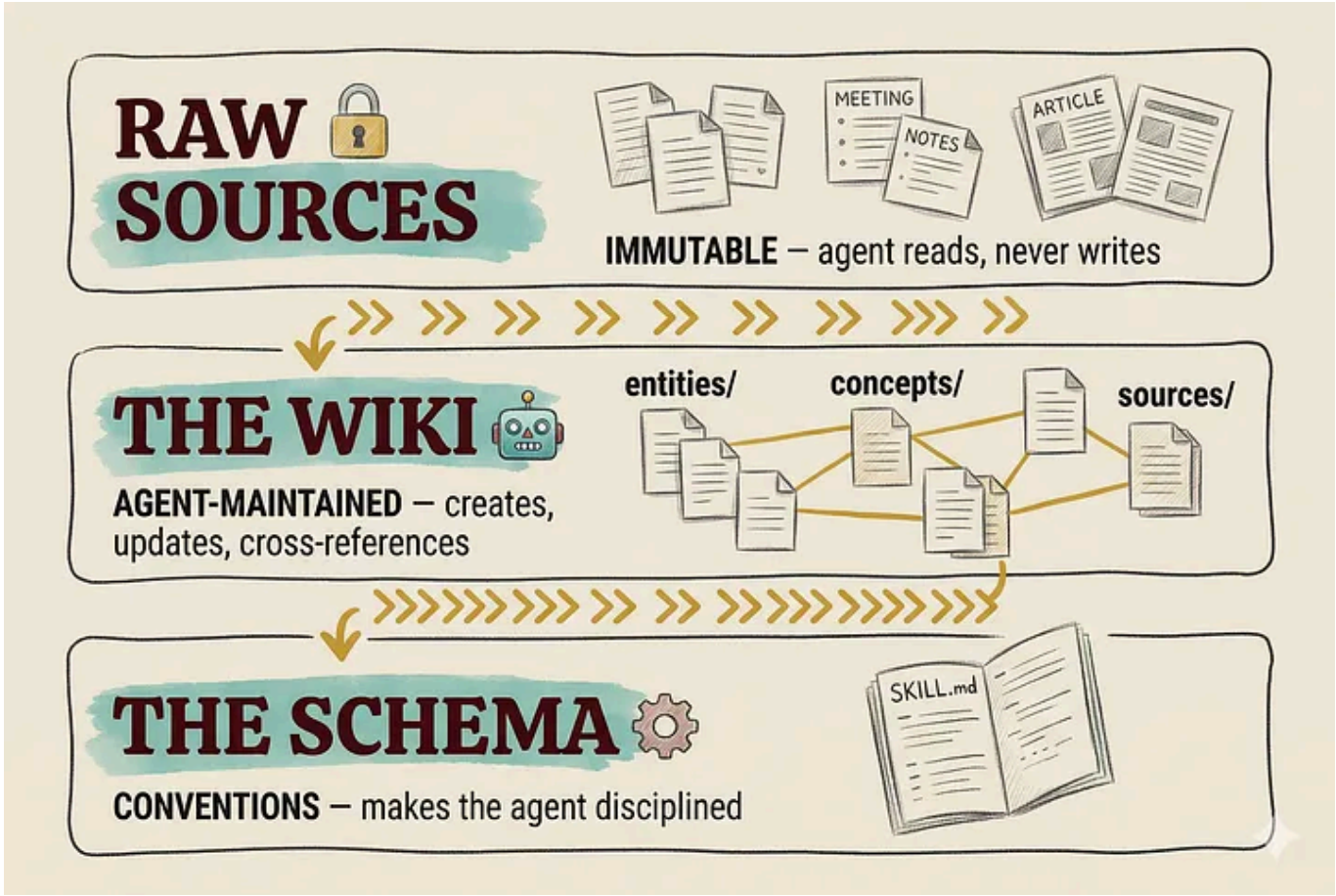
knowledge gets compiled once. Then it stays compiled.

Three layers make this work:

Raw sources — your immutable collection of documents. Articles, meeting notes, architecture decisions, research papers. The LLM reads from these but never modifies them. This is your source of truth.

The wiki — a directory of LLM-generated markdown files. Summaries, entity pages, concept pages, comparisons. The LLM owns this layer entirely. You read it. The LLM writes and maintains it.

The schema — a configuration file (CLAUDE.md or a skill definition) that tells the LLM how the wiki is structured and what workflows to follow. This is what makes the LLM a disciplined wiki maintainer instead of a generic chatbot.



Karpathy's Three-Layer Wiki Architecture | Image: Gemini Pro © Alireza Rezvani

The key insight that gets lost in most coverage: the tedious part of maintaining a knowledge base is not reading or thinking. It is bookkeeping. Updating cross-references. Keeping summaries current.

Noting when new evidence contradicts old claims. Humans abandon wikis because the maintenance burden grows faster than the value. LLMs do not get bored. They do not forget to update a cross-reference. They can touch 15 files in a single pass.

Two Use Cases That Actually Work

The gist is intentionally abstract. Here are two use cases where I have seen the pattern deliver clear value.

Use Case 1: The CTO Decision Wiki

The problem is familiar to anyone leading engineering. Architecture decisions accumulate across months — in meeting transcripts, Slack threads, pull request comments, and hallway conversations. When someone asks "why did we choose X over Y?" six months later, the reasoning is scattered or lost entirely.

The wiki changes this. Create a `raw/` directory and start feeding it the artifacts that contain decisions:

```
raw/
├── meetings/
```

```
│   ├── 2026-01-15-queue-architecture.md
│   ├── 2026-02-03-auth-provider-eval.md
│   └── 2026-03-20-scaling-retrospective.md
├── adrs/
│   ├── adr-007-event-messaging.md
│   └── adr-008-database-migration.md
└── postmortems/
    └── 2026-02-28-notification-outage.md
```

After ingesting these sources, the wiki contains entity pages for each service and vendor, concept pages for architectural patterns you have discussed, and a decision log with cross-references. When the new engineer asks about the notification service, the answer is already compiled — with citations to the specific meeting where the trade-off was debated.

An example entity page after 3 ingests:

```
---
title: "Notification Service"
type: entity
confidence: 0.85
sources:
  - raw/meetings/2026-01-15-queue-architecture.md
  - raw/adrs/adr-007-event-messaging.md
  - raw/postmortems/2026-02-28-notification-outage.md
last_updated: 2026-04-08
```

```
stale: false
---
```

```
# Notification Service

## Architecture
Event-driven via RabbitMQ (chosen over REST polling in January 2026).
Decision rationale: 3x throughput improvement under load testing,
decoupled failure domains. See [[ADR-007-Event-Messaging]].

## Known Issues
- February 2026 outage caused by queue overflow during batch
processing.
  Root cause: missing backpressure configuration. See [[Postmortem-
2026-02-28]].
- Current retry policy is exponential with 5 attempts. Team discussed
  switching to dead-letter queue pattern but deferred to Q2.

## Open Questions
- Should we migrate to NATS for multi-region? Discussed but not
decided
  in [[Meeting-2026-03-20]].
```

Every number has a source. Every decision has context. Open questions are explicit and tracked.

Use Case 2: Technical Content Research Wiki

I write 8–10 Medium articles per month. Each one requires research — reading existing coverage, identifying gaps, collecting evidence. Without a

wiki, that research evaporates after the article ships.

With the wiki pattern, every source I read for every article feeds into the same knowledge base. Concept pages accumulate across articles. When I started researching the LLM Wiki topic for this piece, the wiki already had compiled summaries from my autoresearch and AgentHub research — including Karpathy's design philosophy, his *"idea file"* distribution model, and community responses to both projects.

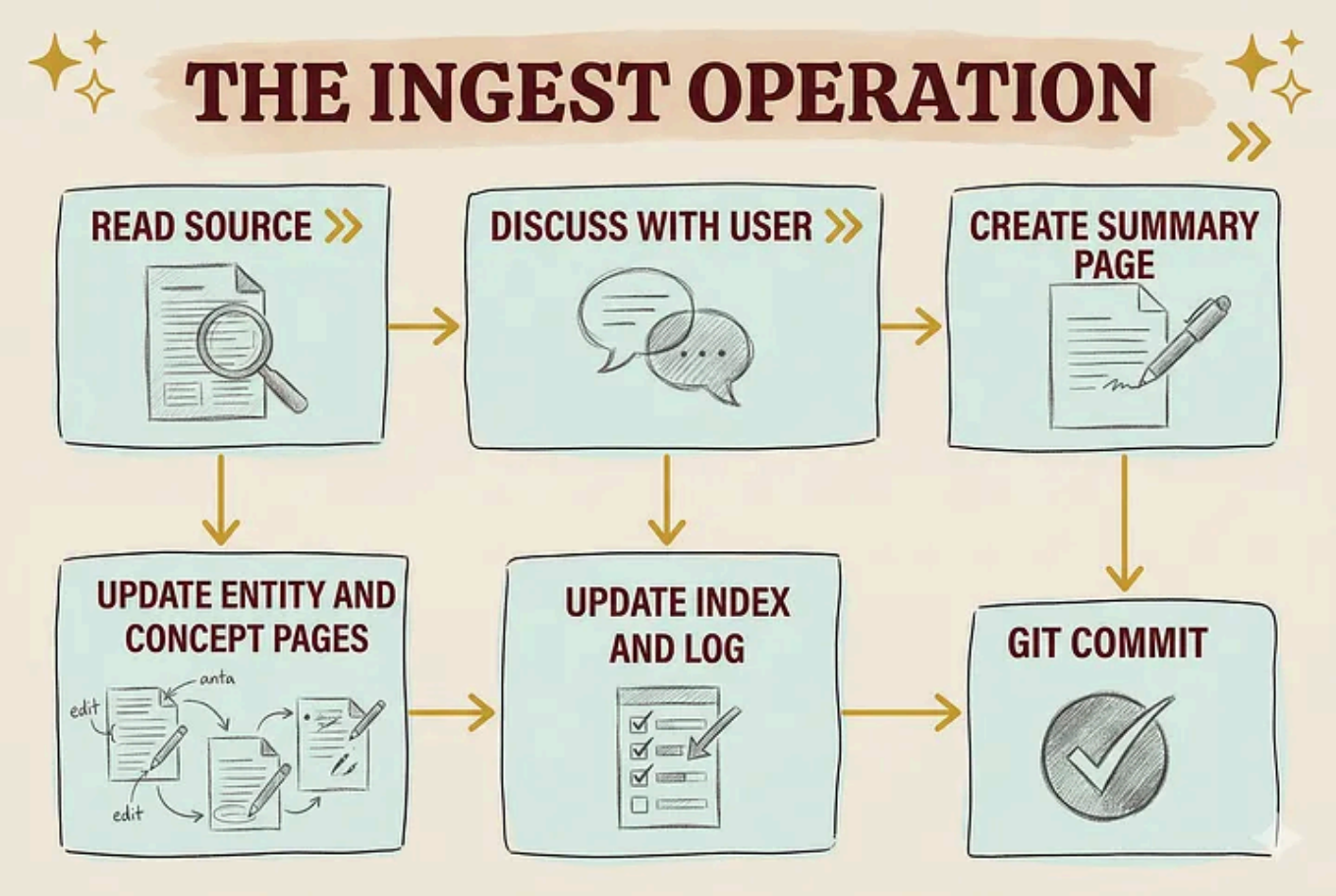
The ingest workflow:

```
# Clip an article to raw/
raw/articles/2026-04-06-mindstudio-llm-wiki-guide.md
```

```
# Ingest it
> /wiki-ingest raw/articles/2026-04-06-mindstudio-llm-wiki-guide.md

# The agent reads the source, creates a summary page,
# updates the existing "LLM Wiki" concept page,
# adds a new "MindStudio" entity page,
# updates the index, appends to the log.
```

After researching five competing articles for this piece, the concept page for *“LLM Wiki”* already contained a synthesis of every approach, where they agree, where they contradict, and which gaps remain. I did not have to re-read anything. The compilation was already done.



The Ingest Operation: 6 Steps From Raw Source to Compiled Knowledge | Image: Gemini Pro © Alireza Rezvani

Building the LLM Wiki Skill: Complete Walkthrough

This is the part that does not exist anywhere else. Karpathy's gist is an idea file — intentionally abstract. The GitHub repos that emerged *(claude-obsidian, MehmetGoekce/llm-wiki, obsidian-wiki)* are implementations, but none of them package the pattern as a reusable Claude Code skill with the structure that makes it installable, portable, and extensible.

Here is the complete skill architecture.

Skill Directory Structure

```
llm-wiki/
├── SKILL.md                    # Main entry — trigger, operations, conventions
├── references/
│   ├── schema.md               # Wiki structure, page types, frontmatter spec
│   ├── page-templates.md       # Templates for every page type
│   └── lint-rules.md           # Health check and contradiction detection
├── commands/
│   ├── wiki-init.md            # Bootstrap a new wiki vault
│   ├── wiki-ingest.md          # Process a source into the wiki
│   ├── wiki-query.md           # Query with synthesis and citations
│   └── wiki-lint.md            # Maintenance and health checks
└── hooks/
    └── session-start.md        # Auto-load context on session start
```

SKILL.md — The Main Entry Point

The SKILL.md file is what Claude Code reads to understand when and how to use the skill. Here is the complete file:

```
---
name: llm-wiki
description: >
  Build and maintain a persistent, compounding knowledge base using
  Karpathy's LLM Wiki pattern. Manages three operations: ingest
  (process new sources into the wiki), query (synthesize answers
  from compiled knowledge), and lint (health-check the wiki for
  contradictions, staleness, and orphans). Works with Obsidian
  as the viewing layer and git for version history.
triggers:
  - /wiki-init
  - /wiki-ingest
  - /wiki-query
  - /wiki-lint
  - User mentions "wiki", "knowledge base", "second brain"
  - User asks about past research, decisions, or accumulated knowledge
context: fork
---
```

```
# LLM Wiki Skill

## Architecture

Three layers, strict separation:
```

```
1. **raw/** — Immutable source documents. You (the human) add files
here.
   The agent reads from raw/ but NEVER writes to it.
2. **wiki/** — Agent-maintained pages. The agent owns this directory
   entirely. It creates, updates, and cross-references pages.
   The human reads wiki/ but should rarely edit it directly.
3. **This skill (schema)** — Conventions and workflows that make
   the agent a disciplined maintainer, not a generic chatbot.

## Operations

### /wiki-init
Bootstrap a new wiki vault. Read `commands/wiki-init.md`.

### /wiki-ingest <path-or-url>
Process a new source into the wiki. Read `commands/wiki-ingest.md`.

### /wiki-query <question>
Query the compiled wiki. Read `commands/wiki-query.md`.

### /wiki-lint
Health-check the wiki. Read `commands/wiki-lint.md`.

## Conventions (Always Follow)

1. **Never modify raw/.** Raw sources are immutable truth.
2. **Always cite sources.** Every claim in a wiki page must
   reference a specific raw source file.
3. **Use [[wikilinks]].** All cross-references use Obsidian
   double-bracket syntax for graph connectivity.
4. **YAML frontmatter on every page.** Title, type, confidence,
   sources, last_updated, stale flag.
5. **Update index.md after every operation.** The index is how
   you (and the agent) navigate the wiki.
6. **Append to log.md after every operation.** Format:
   `## [YYYY-MM-DD] operation | description`
7. **Git commit after every operation.** One commit per
   ingest/query-filed/lint with a descriptive message.
```

Schema: Page Types and Frontmatter

The schema defines what kinds of pages the wiki contains and how they are structured. This goes in `references/schema.md`:

```
# Wiki Schema
```

```
## Page Types

### Source Summary
Created during ingest. One per raw source file.
- Location: wiki/sources/<slug>.md
- Contains: metadata, key claims, relevance assessment
- Links to: concept and entity pages it updates

### Concept Page
Represents an idea, pattern, or domain topic.
- Location: wiki/concepts/<Name>.md
- Contains: definition, evidence from multiple sources,
  open questions, related concepts
- Grows over time as new sources mention the concept

### Entity Page
Represents a specific thing: a person, tool, service, company.
- Location: wiki/entities/<Name>.md
- Contains: attributes, timeline of mentions, relationships
- Updated whenever a new source references the entity

### Comparison Page
Created when the agent or user identifies competing approaches.
- Location: wiki/comparisons/<A>-vs-<B>.md
- Contains: criteria, evidence for each side, verdict if clear
```

```
### Synthesis Page
Created from query results worth preserving.
- Location: wiki/syntheses/<topic>.md
- Contains: the synthesized answer, sources consulted, confidence

## YAML Frontmatter Spec (Required on Every Page)

    ---
    title: "Page Title"
    type: source | concept | entity | comparison | synthesis
    confidence: 0.0-1.0
    sources:
      - raw/path/to/source1.md
      - raw/path/to/source2.md
    last_updated: YYYY-MM-DD
    stale: false
    tags: [tag1, tag2]
    ---

## Special Files

### index.md
Content-oriented catalog. Every page listed with:
- Link, one-line summary, type, confidence, source count.
- Organized by type (sources, concepts, entities, etc.)
- Updated on every ingest operation.

### log.md
Chronological, append-only. Format:
    ## [2026-04-08] ingest | Article: "LLM Wiki Patterns"
    - Created: wiki/sources/llm-wiki-patterns.md
    - Updated: wiki/concepts/RAG.md, wiki/entities/Karpathy.md
    - New page: wiki/concepts/Knowledge-Compilation.md

### overview.md
High-level synthesis of the entire wiki. Updated after every
5th ingest or on explicit request. Captures the current state
of knowledge, major themes, and unresolved questions.
```

Page Templates

These go in `references/page-templates.md` and give the agent exact formats to follow:

```
# Page Templates
```

```
## Source Summary Template

    ---
    title: "[Source Title]"
    type: source
    confidence: 0.9
    sources:
      - raw/[path-to-file].md
    last_updated: YYYY-MM-DD
    stale: false
    tags: []
    ---

    # [Source Title]

    **Author:** [if known]
    **Date:** [publication date]
    **URL:** [if applicable]

    ## Summary
    [3-5 sentence summary of the source's main argument or content]

    ## Key Claims
    - [Claim 1] (confidence: high/medium/low)
```

```
- [Claim 2]
- [Claim 3]

## Relevance
[Why this source matters to the wiki's overall knowledge]

## Pages Updated
- [[Concept-Name]] — [what was added/changed]
- [[Entity-Name]] — [what was added/changed]
```

Concept Page Template

```
---
title: "[Concept Name]"
type: concept
confidence: 0.7
sources:
  - raw/source1.md
  - raw/source2.md
last_updated: YYYY-MM-DD
stale: false
tags: []
---

# [Concept Name]

## Definition
[2-3 sentence definition based on compiled sources]

## Evidence
### From [Source 1 Title]
[Key evidence or perspective from this source]

### From [Source 2 Title]
[Key evidence — note agreements and contradictions]

## Contradictions
[Any conflicting claims between sources, with citations]
```

```
    ## Open Questions
    - [Question not yet answered by any source]
    - [Question where sources disagree]

    ## Related Concepts
    - [[Related-Concept-1]]
    - [[Related-Concept-2]]

---

## Entity Page Template

    ---
    title: "[Entity Name]"
    type: entity
    confidence: 0.8
    sources:
      - raw/source1.md
    last_updated: YYYY-MM-DD
    stale: false
    tags: []
    ---

    # [Entity Name]

    ## Overview
    [Brief description of what/who this entity is]

    ## Key Attributes
    [Relevant attributes compiled from sources]

    ## Timeline
    - [YYYY-MM-DD] [Event or mention] — Source: [[source-page]]

    ## Relationships
    - Connected to [[Entity-2]] via [relationship]
    - Referenced in [[Concept-1]]

    ## Open Questions
    - [Unresolved aspects about this entity]
```

Ingest Command: Step by Step

This is the most critical operation. The file `commands/wiki-ingest.md` contains the exact instructions the agent follows:

```
# /wiki-ingest <source-path-or-url>
```

```
## Workflow

### Step 1: Validate Source
- Confirm the source file exists in raw/ (or download URL to raw/)
- Check if this source has already been ingested (search log.md)
- If already ingested, ask user: "This source was previously
  ingested on [date]. Re-ingest to capture updates?"

### Step 2: Read and Analyze
- Read the full source document
- Identify: key claims, entities mentioned, concepts discussed,
  data points, conclusions, and any claims that contradict
  existing wiki content

### Step 3: Discuss with User
- Present 3-5 key takeaways from the source
- Ask: "Anything you want me to emphasize or skip?"
- Wait for user response before proceeding

### Step 4: Create/Update Pages
- Create a source summary page in wiki/sources/
- For each entity mentioned:
  - If entity page exists: update with new information,
    add source to frontmatter, update confidence
  - If new: create entity page from template
```

```
  - For each concept discussed:
    - If concept page exists: add new evidence section,
      check for contradictions, update confidence
    - If new: create concept page from template
  - If source compares two approaches: consider creating
    a comparison page

  ### Step 5: Update Navigation
  - Update index.md with new/modified pages
  - Append operation to log.md with full details
  - Update overview.md if this is every 5th ingest

  ### Step 6: Commit
  - Git add all changed files
  - Commit message: "ingest: [source title] — [N] pages touched"

  ## Rules
  - NEVER modify files in raw/
  - ALWAYS include source citations in wiki pages
  - If confidence in a claim is below 0.5, flag it explicitly
  - If a new claim contradicts an existing wiki page, add a
    "Contradictions" section — do not silently overwrite
```

Query Command

The `commands/wiki-query.md` file:

```
  # /wiki-query <question>
```

```
  ## Workflow
```

```
### Step 1: Read the Index
- Read wiki/index.md to identify relevant pages
- Select pages most likely to contain the answer

### Step 2: Read Relevant Pages
- Read selected wiki pages (not raw sources)
- If wiki pages reference raw sources for critical claims,
  verify against the raw source

### Step 3: Synthesize Answer
- Compile an answer from multiple wiki pages
- Include citations: "According to [[Page-Name]], ..."
- Note confidence level and any contradictions

### Step 4: Offer to File
- Ask: "Should I save this answer as a synthesis page?"
- If yes: create wiki/syntheses/<topic>.md
- Update index.md and log.md

## Rules
- Always cite which wiki pages you drew from
- If the wiki does not contain enough information to answer,
  say so — do not fall back to general knowledge
- Prefer wiki content over your training data
```

Lint Command

The `commands/wiki-lint.md` file:

```
# /wiki-lint
```

```
## Checks to Run

1. **Orphan Detection**
   - Find pages with no inbound [[wikilinks]]
   - Suggest connections or flag for review

2. **Staleness Check**
   - Flag pages where last_updated > 30 days
   - Flag pages where stale: true

3. **Contradiction Scan**
   - Compare claims across pages on the same topic
   - Flag where sources disagree without acknowledgment

4. **Confidence Audit**
   - List pages with confidence < 0.5
   - Suggest sources that could improve confidence

5. **Missing Pages**
   - Find [[wikilinks]] that point to non-existent pages
   - Suggest creating them

6. **Index Sync**
   - Verify every wiki page appears in index.md
   - Verify no index entries point to deleted pages

## Output
- Print a health report with counts for each check
- List specific pages needing attention
- Suggest new sources to look for based on gaps
- Append lint results to log.md
```

Session Start Hook

The `hooks/session-start.md` file ensures the agent has context on every session:

```
# Session Start Hook
```

```
When a session begins in a directory containing a wiki:

1. Read wiki/index.md (full file)
2. Read the last 10 entries of wiki/log.md
3. Note the total page count and last operation date
4. Be ready to answer questions about wiki contents
   without the user needing to specify paths
```

Installation

Copy the skill to your Claude Code skills directory and point it at an Obsidian vault:

```
# Clone or copy the skill
cp -r llm-wiki/ ~/.claude/skills/llm-wiki/
```

```
# Open Claude Code in your Obsidian vault
cd ~/Documents/ObsidianVault
claude

# Bootstrap the wiki
> /wiki-init
```

```
# Start ingesting sources
> /wiki-ingest raw/articles/first-article.md
```

The `/wiki-init` command creates the full directory structure, initializes git tracking, and creates empty `index.md`, `log.md`, and `overview.md` files. From there, every ingest builds the wiki incrementally.

You can also download the full version as a plugin or skill from my agent skills repo for free, below:

claude-skills/engineering/llm-wiki at main · alirezarezvani/claude-skills

232+ Claude Code skills & agent plugins for Claude Code, Codex, Gemini CLI, Cursor, and 8 more coding agents …

github.com

What Breaks

The pattern is powerful. It is not without friction.

Scale ceiling. The index-based approach — where the agent reads `index.md` to find relevant pages — works well up to roughly 200 pages and 100 sources.

Beyond that, the index itself becomes too large for efficient scanning. The mitigation is adding semantic search. Tobi Lutke's `qmd` tool provides hybrid BM25/vector search over markdown files with an MCP server interface. But for most personal wikis, you will not hit this ceiling for months.

Hallucination during compilation. The LLM can misinterpret a source during ingest. I have seen it attribute a claim to the wrong meeting, or merge two similar but distinct concepts into one page. The mitigation is strict citation enforcement — every claim must reference a specific raw source file — and regular lint passes that surface contradictions. The raw sources remain immutable, so you can always verify.

Context window pressure. A single ingest of a long document (a 10,000-word research paper, a full meeting transcript) can push against token limits, especially when the agent also needs to read existing wiki pages to check for updates. The mitigation is the skill's design: the agent reads the index first (compact), identifies which pages to update (selective), and only loads those specific pages.

Single-agent writes. If two agents (or two Claude Code sessions) write to the wiki simultaneously, you get merge conflicts. Karpathy's gist does not solve this. For solo use, this is a non-issue. For team use, you need coordination —

one agent session at a time, or a pull-request review pattern where the agent proposes changes and a human approves.

Maintenance discipline. The wiki only compounds if you feed it. Unlike a static notes app where neglect just means your notes age, a neglected wiki develops staleness that the next ingest has to reconcile. Build the habit of ingesting sources as you encounter them, not in batch.

What I Am Still Figuring Out

When does a personal wiki become a team wiki? The schema changes are obvious — you add contributor tracking, review gates, access controls. But the harder question is trust. A personal wiki can tolerate 0.7 confidence on a concept page because you know the context. A team wiki needs higher standards because someone else will read it without your background.

Could the lint operation run autonomously? An agent that health-checks the wiki nightly, surfaces contradictions, and suggests sources to fill gaps — without a human in the loop. The pattern supports it. The trust model does not, yet.

And the question I keep returning to: are we building tools that make us think better, or tools that think for us? The wiki pattern sits at an interesting boundary. The agent does the bookkeeping. But the human still decides what to ingest, what questions to ask, and what the compiled knowledge means. That division of labor feels right. I am watching to see if it holds.

Frequently Asked Questions About LLM Wiki and Claude Code

What is Karpathy's LLM Wiki and how does it differ from RAG?

The LLM Wiki is a persistent knowledge base where an LLM compiles and maintains structured markdown files, rather than retrieving raw documents at query time. In a RAG system, the model re-discovers relevant information from scratch on every question. In the wiki pattern, knowledge is compiled once during ingest — summaries written, cross-references maintained, contradictions flagged — and then stays compiled. The wiki grows richer with every source added. RAG stays stateless.

How do I set up an LLM Wiki with Claude Code and Obsidian?

Install the llm-wiki Claude Code skill, open Claude Code in your Obsidian vault directory, and run /wiki-init to bootstrap the structure. The skill

creates the three-layer architecture (raw sources, wiki pages, schema), initializes git tracking, and sets up the index and log files. From there, drop source files into `raw/` and run `/wiki-ingest` to process each one. Obsidian provides the viewing layer with graph view, backlinks, and live editing — Claude Code does the compilation and maintenance.

Can I use the LLM Wiki pattern for team knowledge management?

Yes, but you need coordination mechanisms that the basic pattern does not include. For solo use, one Claude Code session at a time avoids write conflicts. For teams, add a review queue — the agent proposes wiki updates as git branches, and a human reviews before merging. YAML frontmatter with confidence scores and source citations gives reviewers the context they need to approve or reject changes. Several community implementations (AI-Context-OS, Continuity) have added multi-user coordination layers.

How large can an LLM Wiki grow before performance degrades?

The index-based approach works well up to approximately 200 pages and 100 sources. At that scale, the agent reads the full index to locate relevant pages. Beyond that, you need a search layer — `qmd` provides hybrid BM25/vector search with an MCP server, or you can build a simpler grep-based search script. The wiki pattern itself scales indefinitely; the limiting factor is how you retrieve relevant pages for the agent to read.

Building systems that think with you, not for you? I write about Claude Code, agentic workflows, and practical AI development every week.

Subscribe to the newsletter →

About the Author

Reza is a Berlin-based CTO who leads a 7-person engineering team and builds AI development systems in production. He has written about every major Karpathy release — from autoresearch to AgentHub — with hands-on implementation guides and reusable Claude Code skills.

Connect:

Website: alirezarezvani.com
LinkedIn: linkedin.com/in/alirezarezvani
Newsletter: claude-code.beehiiv.com

[Read more on Medium →]

Claude Code + | Anthropic Claude + | Productivity + | LLM +

Ai Knowledge Management +



Written by Reza Rezvani

7.5K followers · 91 following

Follow

CTO & AI builder based in Berlin. Writing about Claude Code, agentic workflows, and shipping real products with AI. 20+ years of turning ideas into products.

Responses (5)



Krzysztof

What are your thoughts?



Reza Rezvani Author
Apr 12

The skill files are available in my claude-skills repo on GitHub. The part that surprised me most was how fast the wiki becomes useful — after just 5-6 sources, the cross-references start connecting things you did not realize were related.

If you are... more

4 Reply



Kamrun Nahar she/her
Apr 18

following you after this one ✨...

3 1 reply Reply



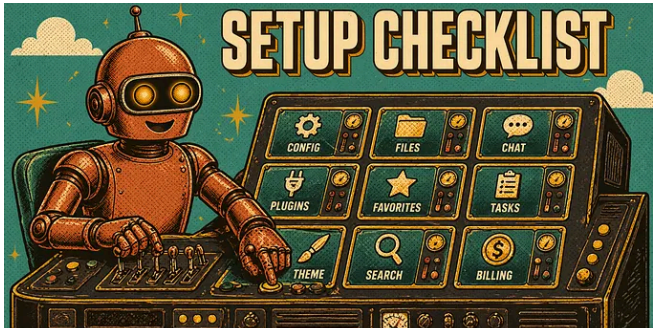
Cyril Simonnet
Jul 24

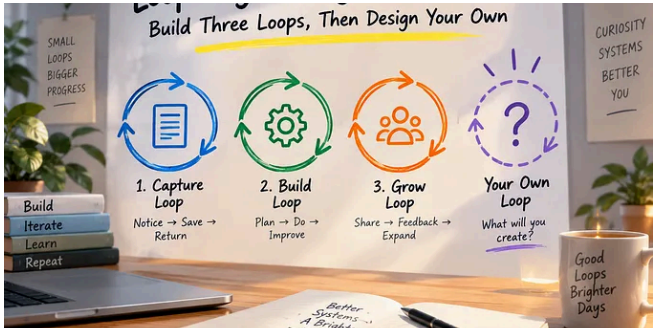
Compiling knowledge instead of retrieving it is the right framing, and it changes how you design the whole vault. Once you treat Claude Code as the writer (not just the search layer), the folder structure, frontmatter schema, and linking conventions... more

1 Reply

See all responses

More from Reza Rezvani





In AI Coding Tools by Reza Rezvani · Apr 30

The Complete Claude Setup Checklist: 72 Steps From Default ...

In Claude and Claude Code Every setting, folder, connector, and workflow trick that...

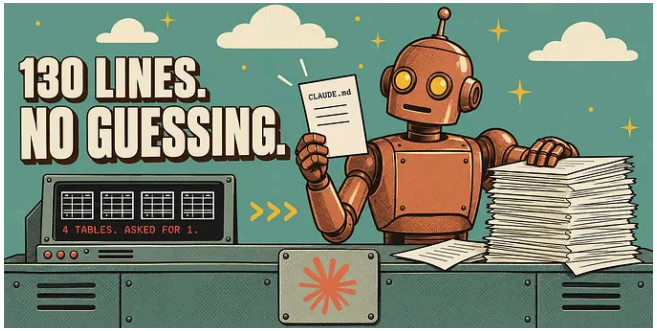
187 1 1

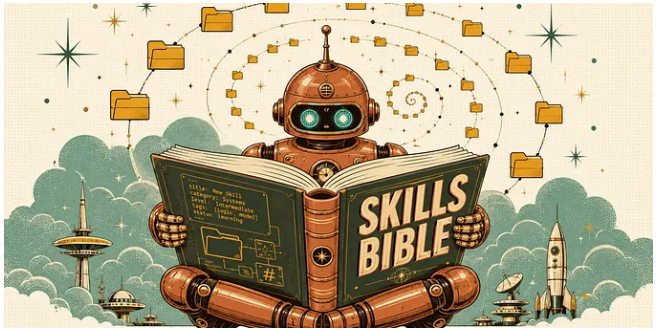
Reza Rezvani · Sep 14

Loop Engineering in Practice: Build Three Loops, Then Design Your...

The primitives all ship inside Claude Code now. Build three real loops in your own repo...

7 1





Reza Rezvani · Sep 6

A global CLAUDE.md and its best pieces

In AI Coding Tools by Reza Rezvani · May 11

Claude Skills Bible: Lessons from 235 Production Skills

The 130 lines that stopped Claude from guessing on my behalf, and the prompts to...

32

A practitioner manual for Claude.ai and Claude Code: the patterns, anti-patterns, an...

105 2 1

See all from Reza Rezvani

Help Status About Careers Press Blog Store Privacy Rules Terms Text to speech